AGENTES DE IA · 2026-2

PONYARCHIVE

Biblioteca inteligente de **My Little Pony: Friendship Is Magic**
con agente RAG, LangGraph y modelo local

Integrante: **Mariana Lemus Esguerra** · Trabajo individual

Presentación: proyecto individual. PonyArchive responde preguntas sobre la serie usando solo documentos indexados y rechaza explícitamente cuando no tiene evidencia.

// Problema o necesidad

Un LLM responde **de memoria**: suena seguro incluso cuando no sabe.

FUNDAMENTAR

Cada respuesta debe salir de los documentos del corpus y citar su fuente — no de la memoria paramétrica del modelo.

RECHAZAR

Fuera del corpus, decir «no lo sé» con la frase literal exigida, en vez de inventar algo plausible.

> «No tengo suficiente información en mi base de conocimiento para responder esa pregunta.»

Caso adicional: **conocimiento de contexto del aula** — datos que constan en una nota propia, no en ningún documento público, deben responderse citando esa nota.

Dos exigencias del enunciado: respuestas fundamentadas y rechazo explícito. Y un caso real del profesor: preguntas de contexto de clase.

// Arquitectura

FRONTEND
React + Vite + TS
chat · documentos · historial

->

BACKEND FASTAPI
17 endpoints · JWT
grafo LangGraph

->

CHROMADB + SQLITE
3.100 fragmentos
usuarios y conversaciones

->

OLLAMA (LOCAL)
qwen3.5:4b
RTX 4070 · 67 tok/s

SIN NUBE DE PAGO

Todo local o free tier: presupuesto **0 USD**.

PRIVACIDAD

Ninguna pregunta ni documento sale del equipo.

DOCKER

`docker compose up -d` levanta backend (8010) y frontend (8081).

Tres piezas desacopladas más el modelo local. En la nube: frontend Vercel, backend Render, Qdrant Cloud y OpenAI en free tier.

// Tecnologías usadas

CAPA	TECNOLOGÍA	POR QUÉ
Backend	Python 3.14 · FastAPI · Uvicorn	Encaja con LangChain; OpenAPI en `/docs`
Agente	LangChain + LangGraph	Grafo de estados en código, sin constructores visuales
LLM	Ollama — `qwen3.5:4b`	Local, gratuito, 67 tok/s en RTX 4070
Embeddings	`multilingual-e5-small` (384 d)	Multilingüe: rinde en español
Vectorial	ChromaDB persistente	Embebida, sin servidor externo
Frontend	React + Vite + TypeScript	Ligero; build con `tsc` + vite sin errores
Datos	Pony API + Fandom (CC BY-SA)	243 episodios · 555 personajes · 192 canciones · prosa de Fandom

Secretos en `.env` (nunca en el repo) · código bajo licencia MIT · dependencias fijadas.

Ningún servicio comercial de pago. Si preguntan por el modelo: 4B por VRAM y latencia; el RAG hace el trabajo pesado.

// Agente con LangChain y LangGraph

VALIDATE_INTENT
dominio · catálogo · fuera de dominio

->

RETRIEVE
Chroma filtrada por usuario

->

EVALUATE_SUFFICIENCY
¿hay evidencia?

GENERATE
prompt restringido + citas [n]

->

VALIDATE_ANSWER
entidades en el contexto

->

PERSIST
historial del usuario

Rutas condicionales: sin evidencia → **reject** (frase literal) · consultas de catálogo → SQL directo sin LLM.

```
 g = StateGraph(AgentState)
g.add_node("retrieve", retrieve)
g.add_conditional_edges("evaluate_sufficiency",
    lambda s: "generate" if s["sufficient"] else "reject")
```

Todo en Python. Las 7 estrategias anti-alucinación viven en estos nodos: umbral, suficiencia, prompt, rechazo, fuentes, validación y SQL.

// Base vectorial y carga de documentos

CARGAR	->	TROCEAR	->	EMBEBER	->	INDEXAR
Pony API · semillas · .txt/.md del usuario		fragmentos con solapamiento		multilingual-e5-small · "passage: …"		Chroma + metadatos

METADATOS POR FRAGMENTO

`doc_type`, `title`, `user_id`, `themes`, `chunk_index` → cada fuente reconstruye su documento original.

IDEMPOTENTE Y AISLADO

Deduplicación por `content_hash` (por usuario); las cargas de un usuario no se ven desde otro.

Consulta: `TOP_K=6` · umbral `0.62` (e5 local) / `0.50` (nube OpenAI, `0.45` para la nota del curso) · bonificación +0,05 a los documentos propios recién subidos.

Estado actual: 3.100 fragmentos (1.015 documentos: 243 episodios, 555 personajes con descripción de Fandom, 192 canciones, fichas narrativas, subidos y notas), CC BY-SA. Formatos de carga: la interfaz (la que se usa en la demo) admite .txt y .md. El endpoint API también acepta .csv/.pdf/.docx (pypdf/python-docx), pero en la demo se sube un .txt.

// Seguridad y usuarios

AUTENTICACIÓN

- Registro / login / refresh / logout
- Contraseñas con hash · JWT (HS256)
- Sin token en endpoints de negocio → **401**

AISLAMIENTO DE DATOS

- Historial por usuario (caso B04)
- Documentos propios vs. compartidos (T10/T10b)
- Cada búsqueda filtra por visibilidad

CONFIGURACIÓN

- CORS con lista explícita de orígenes
- `.env` fuera del repo · `.env.example` documentado
- Contenedor sin privilegios

JWT PROPIO (PLAN PROPIO)

Autenticación propia (HS256) en lugar de Keycloak: el enunciado §2.1 permite implementarla por cuenta propia; coste 0 USD y control total.

R-04: la frase "ningún endpoint sin autenticación" choca con /health, OPTIONS y las rutas de login; está documentado en el documento técnico.

// Demostración

RESPONDIDO

«¿Qué ocurre en Lesson Zero?» → respuesta con **6 fuentes** y similitudes.

RECHAZADO

«¿Cuánto sulfato de cobre…?» → frase literal de rechazo.

DOC NUEVO

Se sube un .txt y la siguiente pregunta responde desde ese archivo.

CONTEXTO DEL CURSO

«¿A Duvan Gomez le pega la esposa?» → sí, con la nota como fuente (sim 0,905 local / 0,598 nube).

AISLAMIENTO

Dos usuarios: cada uno solo ve y consulta sus documentos.

PRUEBAS

12/12 SMOKE

E2E DOCKER OK

Recorrido en vivo: login → chat con fuentes → rechazo → carga de documento → grafo en el código → despliegue.

Orden de la demo: arquitectura, login, chat correcto, rechazo, carga de documento + pregunta, grafo en código, evidencia de despliegue. 7-10 minutos.

// Conclusiones

1. Requisitos mínimos cumplidos: backend + frontend, auth, agente con LangGraph, **modelo local**, base vectorial con carga de documentos, respuestas fundamentadas y **rechazo explícito**.

2. 7 estrategias anti-alucinación (mínimo 2 exigidas), verificadas con **12 pruebas automatizadas** y un E2E completo sobre Docker.

3. LangGraph hace auditables las decisiones del agente: cada ruta es una arista en código, sin constructores visuales.

4. Un modelo de 4B basta cuando el RAG hace el trabajo pesado: la calidad depende de la recuperación y del prompt, no del tamaño del modelo.

5. Coste 0 USD: pila local completa y despliegue en free tier (Vercel, Render, Qdrant Cloud y cupo gratuito de OpenAI).

Repo: `github.com/navyri/PonyArchive` · Documento técnico con sus 14 secciones · Licencias: código MIT · datos CC BY-SA (Pony API y Fandom).

Cierre: preguntas del público. Repo y documento técnico como evidencia.

FIN DE LA PRESENTACIÓN

GRACIAS POR SU ATENCIÓN

PonyArchive · biblioteca inteligente con agente RAG
preguntas, comentarios y demo en vivo

Mariana Lemus Esguerra · github.com/navyri/PonyArchive

Agradecimiento final. Se abre el turno de preguntas; el repo y el documento técnico quedan como evidencia.